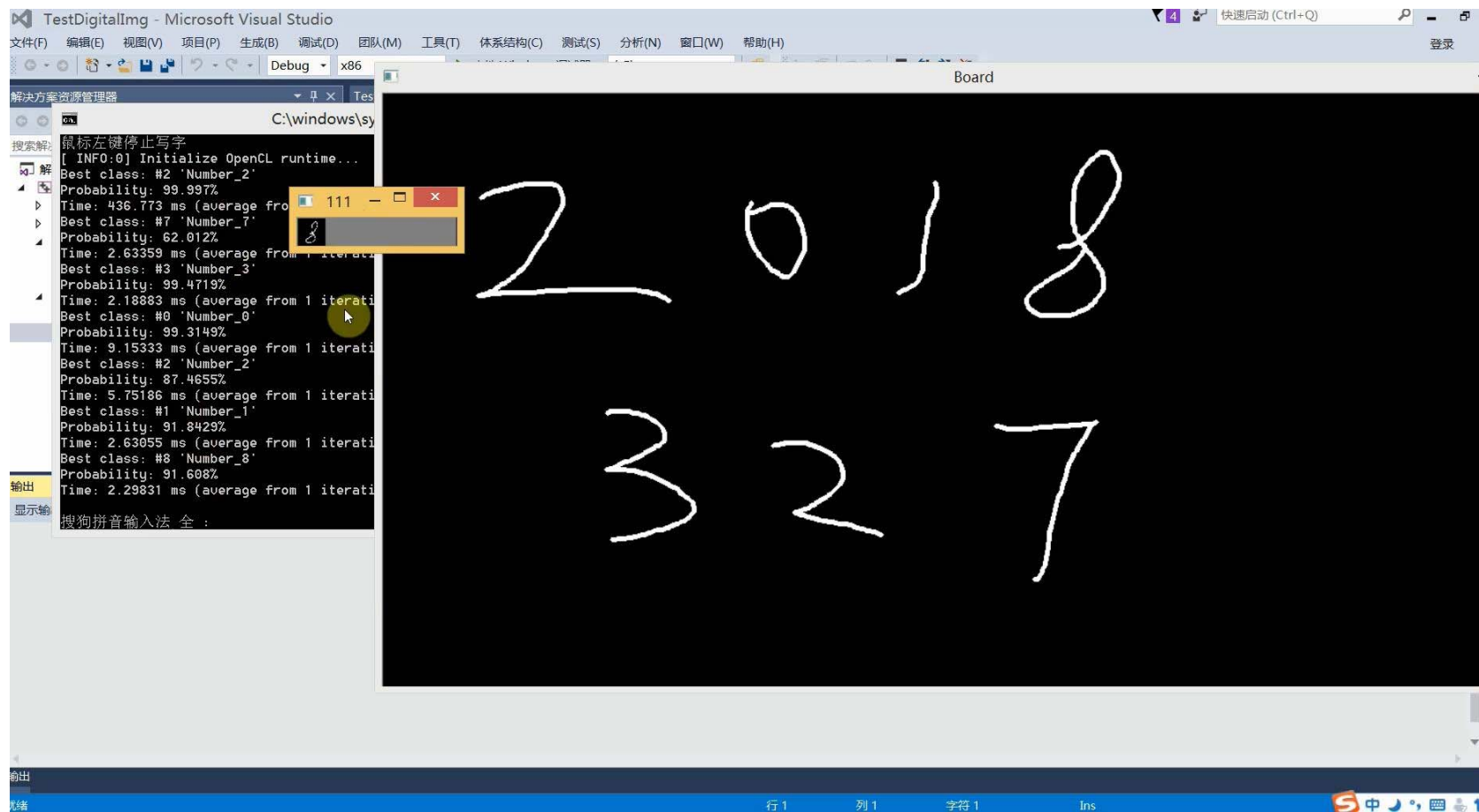


基于深度学习的手写字 识别系统

文嘉俊

计算机视觉研究所
深圳大学
2018. 7



Demc

- Caffe简介与安装
- 手写字识别深度模型训练与测试
- 结合OpenCV的手写字识别系统

- Caffe简介与安装
- 手写字识别深度模型训练与测试
- 结合OpenCV的手写字识别系统

- Caffe简介与安装

1. 由伯克利视觉和学习中心开发的，基于C++/GUDA/Python实现的卷积神经网络框架。
2. 全 称： Convolutional Architecture for Fast Feature Embedding

<http://caffe.berkeleyvision.org>

- Caffe简介与安装

Caffe 特点:

- 实现了前馈神经网络框架**CNN**，而不是递归网络框架**RNN**；
- 速度快；
- 适合做特征提取，实际上适合做二维图像数据的特征提取；
- **Caffe** 完全开源；
- **Caffe** 提供一整套工具集，可用于模型训练、预测、微调、发布、数据预处理，以及良好的自动测试；
- **Caffe** 带有一系列参考模型和快速上手例程；
- **Caffe**在国内外有比较活跃的社区，有很多衍生项目，如 Caffe for Windows, Caffe with OpenCL, NVIDIA DIGITS2, R-CNN 等
- **Caffe** 代码组织良好，可读性强。

● Caffe简介与安装

下载Windows版本Caffe

<https://github.com/microsoft/caffe>

GitHub, Inc. [US] | <https://github.com/microsoft/caffe>

设置 Home - Springer ScienceDirect IEEE Xplore Digital 电子资源 | 深圳大学 ezproxy.lib.szu.edu 投稿

Features Business Explore Marketplace Pricing This repository Search Sign in or Sign up

Microsoft / **caffe**
forked from BVLC/caffe

Watch 143 Star 904 Fork 15,003

Code Issues 75 Pull requests 3 Projects 0 Insights

Join GitHub today
GitHub is home to over 20 million developers working together to host and review code, manage projects, and build software together.
[Sign up](#)

Caffe on both Linux and Windows

3,842 commits 2 branches 11 releases 209 contributors

Branch: master New pull request Find file Clone or download

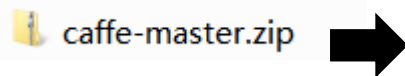
This branch is 107 commits ahead, 402 commits behind BVLC:master. #146 Compare

zer0n Update README.md Latest commit ed05614 on 7 Dec 2016

cmake	Add c++11 flag to cuda cmake build	2 years ago
data	Merge pull request #4455 from ShaggO/spaceSupportILSVRC12MNIST	2 years ago
docker	Update Dockerfile to cuDNN v5	2 years ago
docs	Update supported cuDNN version in the documentation	2 years ago
examples	Merge branch 'master' of ssh://github.com/BVLC/caffe into bvlc	2 years ago

● Caffe简介与安装

假设放到路径C:\DeepLearning



名称	修改日期	类型	大小
cmake	2018/6/14 11:39	文件夹	
data	2018/6/14 11:39	文件夹	
docker	2018/6/14 11:39	文件夹	
docs	2018/6/14 11:39	文件夹	
examples	2018/6/14 11:39	文件夹	
include	2018/6/14 11:39	文件夹	
matlab	2018/6/14 11:39	文件夹	
models	2018/6/14 11:39	文件夹	
python	2018/6/14 11:39	文件夹	
scripts	2018/6/14 11:39	文件夹	
src	2018/6/14 11:40	文件夹	
tools	2018/6/14 11:40	文件夹	
windows	2018/6/14 11:40	文件夹	
.Doxyfile	2016/12/6 12:41	DOXYFILE 文件	100 KB
.gitattributes	2016/12/6 12:41	GITATTRIBUTES ...	3 KB
.gitignore	2016/12/6 12:41	GITIGNORE 文件	2 KB
.travis.yml	2016/12/6 12:41	YML 文件	2 KB
appveyor.yml	2016/12/6 12:41	YML 文件	1 KB
caffe.cloc	2016/12/6 12:41	CLOC 文件	2 KB
CMakeLists.txt	2016/12/6 12:41	文本文档	3 KB
CONTRIBUTING.md	2016/12/6 12:41	MD 文件	2 KB
CONTRIBUTORS.md	2016/12/6 12:41	MD 文件	1 KB
INSTALL.md	2016/12/6 12:41	MD 文件	1 KB
LICENSE	2016/12/6 12:41	文件	3 KB
Makefile	2016/12/6 12:41	文件	24 KB
Makefile.config.example	2016/12/6 12:41	EXAMPLE 文件	5 KB
README.md	2016/12/6 12:41	MD 文件	5 KB

● Caffe简介与安装

- 安装Visual Studio 2013 版。由于目前只需编译CPU模式Caffe，故暂不需要安装CUDA Toolkit 7.5和cuDNN。
- 进入C:\DeepLearning\caffe-master\windows目录，将文件CommonSettings.props.example重命名为CommonSettings.props，修改其内容如下：

CommonSettings - 写字板

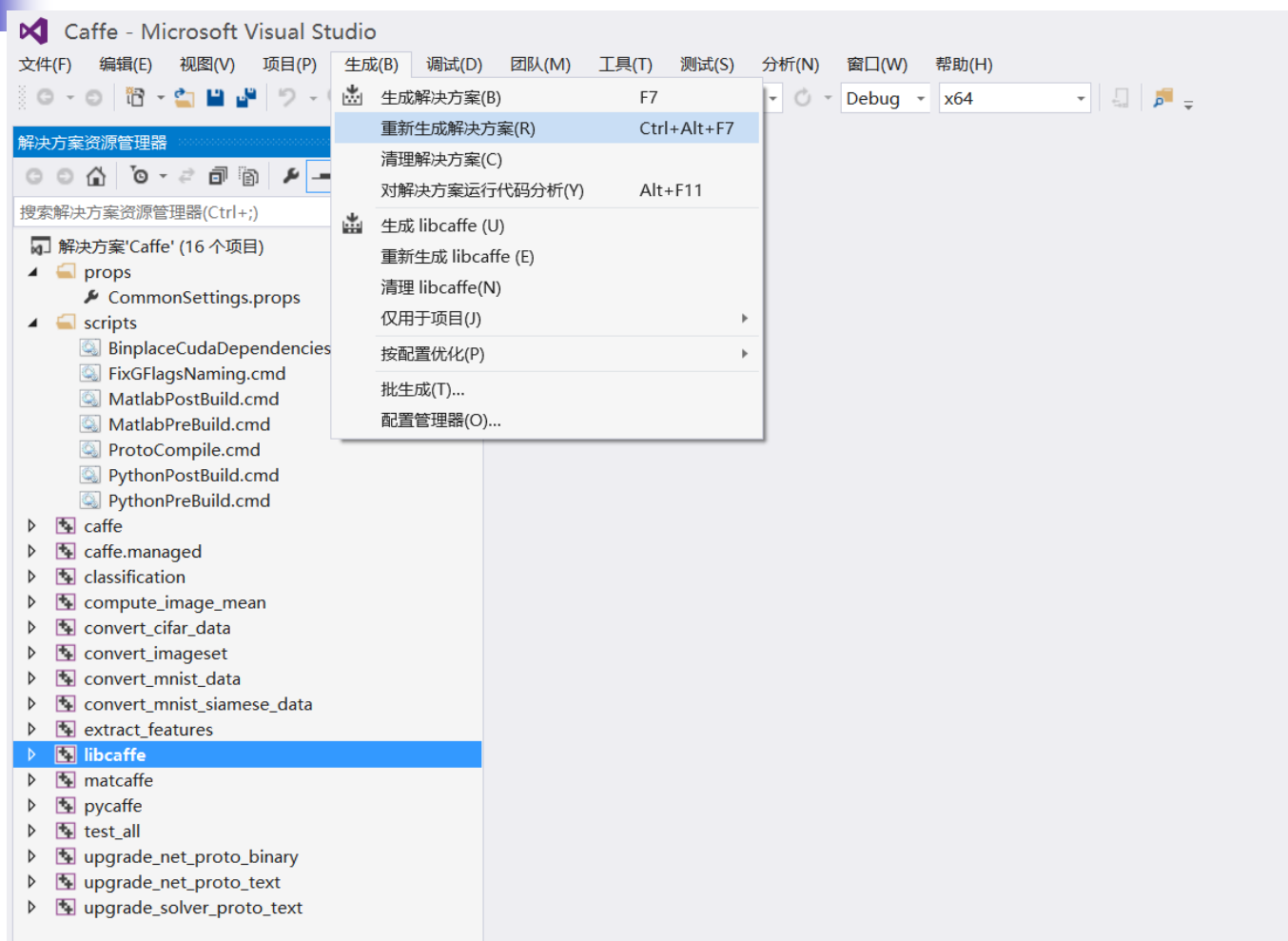
```
<?xml version="1.0" encoding="utf-8"?>
<Project ToolsVersion="4.0"
xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <ImportGroup Label="PropertySheets" />
  <PropertyGroup Label="UserMacros">
    <BuildDir>$(SolutionDir)..\Build</BuildDir>
    <!--NOTE: CpuOnlyBuild and UseCuDNN flags can't be set at the same
time.-->
    <CpuOnlyBuild>true</CpuOnlyBuild>
    <UseCuDNN>>false</UseCuDNN>
    <CudaVersion>7.5</CudaVersion>
    <!-- NOTE: If Python support is enabled, PythonDir (below) needs to be
      set to the root of your Python
installation
      does not contain debug libraries, debug build will not work. -->
    <PythonSupport>>false</PythonSupport>
```

● Caffe简介与安装

- 修改完成后，保存该文件。双击同一目录下的Caffe.sln文件，代开Windows Caffe 工程。
- 单击菜单“生成” “重新生成解决方案”确定“Debug”或“Release”，开始漫长的编译过程。

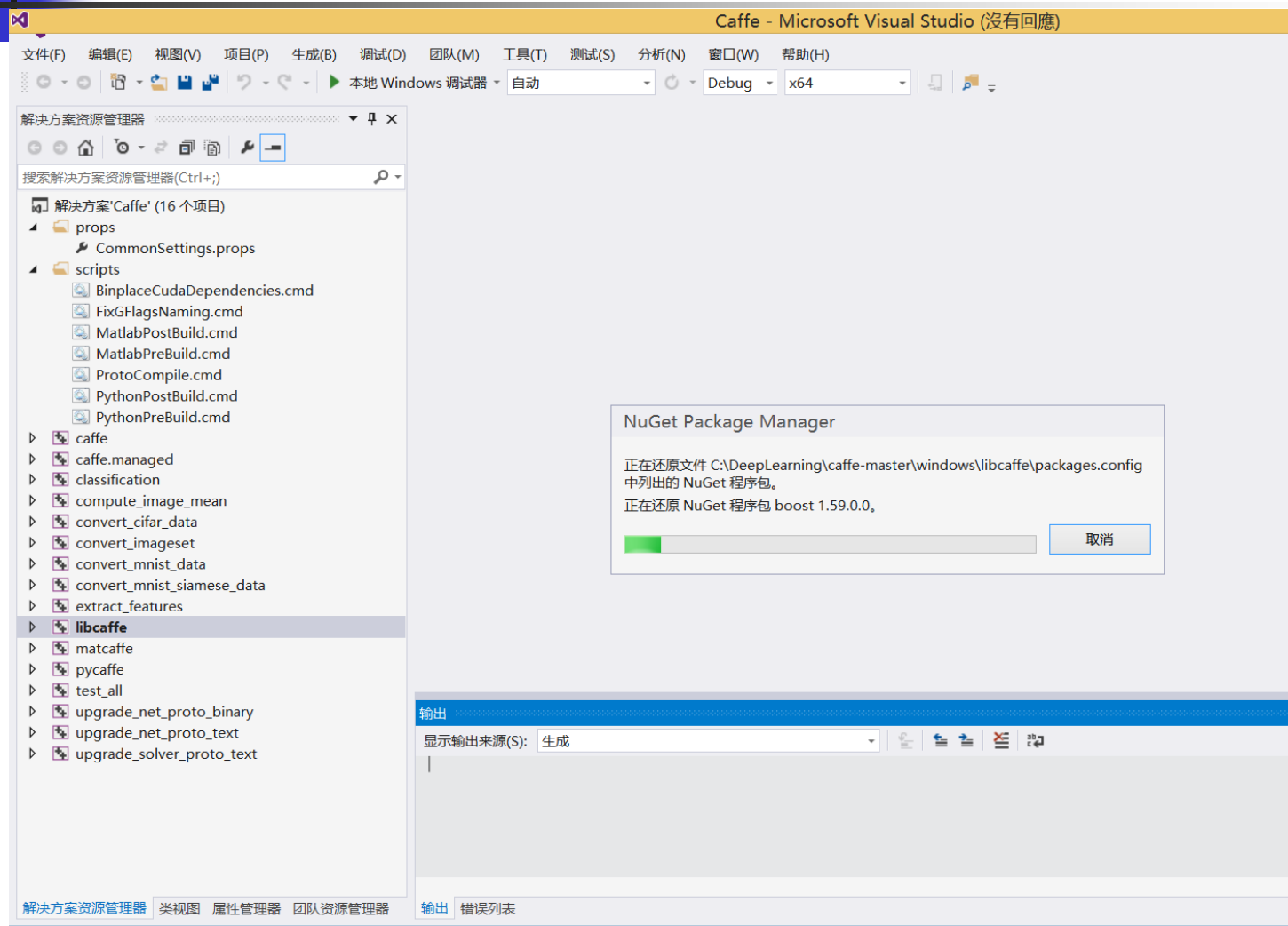
windows			
查看			
这台电脑 > Windows8_OS (C:) > DeepLearning > caffe-master > windows			
名称	修改日期	类型	大小
caffe	2018-06-14 11:50	文件夹	
caffe.managed	2018-06-14 11:50	文件夹	
classification	2018-06-14 11:50	文件夹	
compute_image_mean	2018-06-14 11:50	文件夹	
convert_cifar_data	2018-06-14 11:50	文件夹	
convert_imageset	2018-06-14 11:50	文件夹	
convert_mnist_data	2018-06-14 11:50	文件夹	
convert_mnist_siamese_data	2018-06-14 11:50	文件夹	
extract_features	2018-06-14 11:50	文件夹	
libcaffe	2018-06-14 11:50	文件夹	
matcaffe	2018-06-14 11:50	文件夹	
pycaffe	2018-06-14 11:50	文件夹	
scripts	2018-06-14 11:50	文件夹	
test_all	2018-06-14 11:50	文件夹	
upgrade_net_proto_binary	2018-06-14 11:50	文件夹	
upgrade_net_proto_text	2018-06-14 11:50	文件夹	
upgrade_solver_proto_text	2018-06-14 11:50	文件夹	
Caffe	2018-06-14 12:02	SQL Server Com...	64 KB
Caffe	2016-12-06 12:41	Microsoft Visual ...	10 KB
CommonSettings	2018-06-14 12:01	Project Property...	6 KB
CommonSettings.targets	2016-12-06 12:41	TARGETS 文件	1 KB
nuget	2016-12-06 12:41	XML Configurati...	1 KB

● Caffe简介与安装



必须要连接网络状态下操作

● Caffe简介与安装



通过网络获取依赖包（保持网络畅通）

● Caffe简介与安装

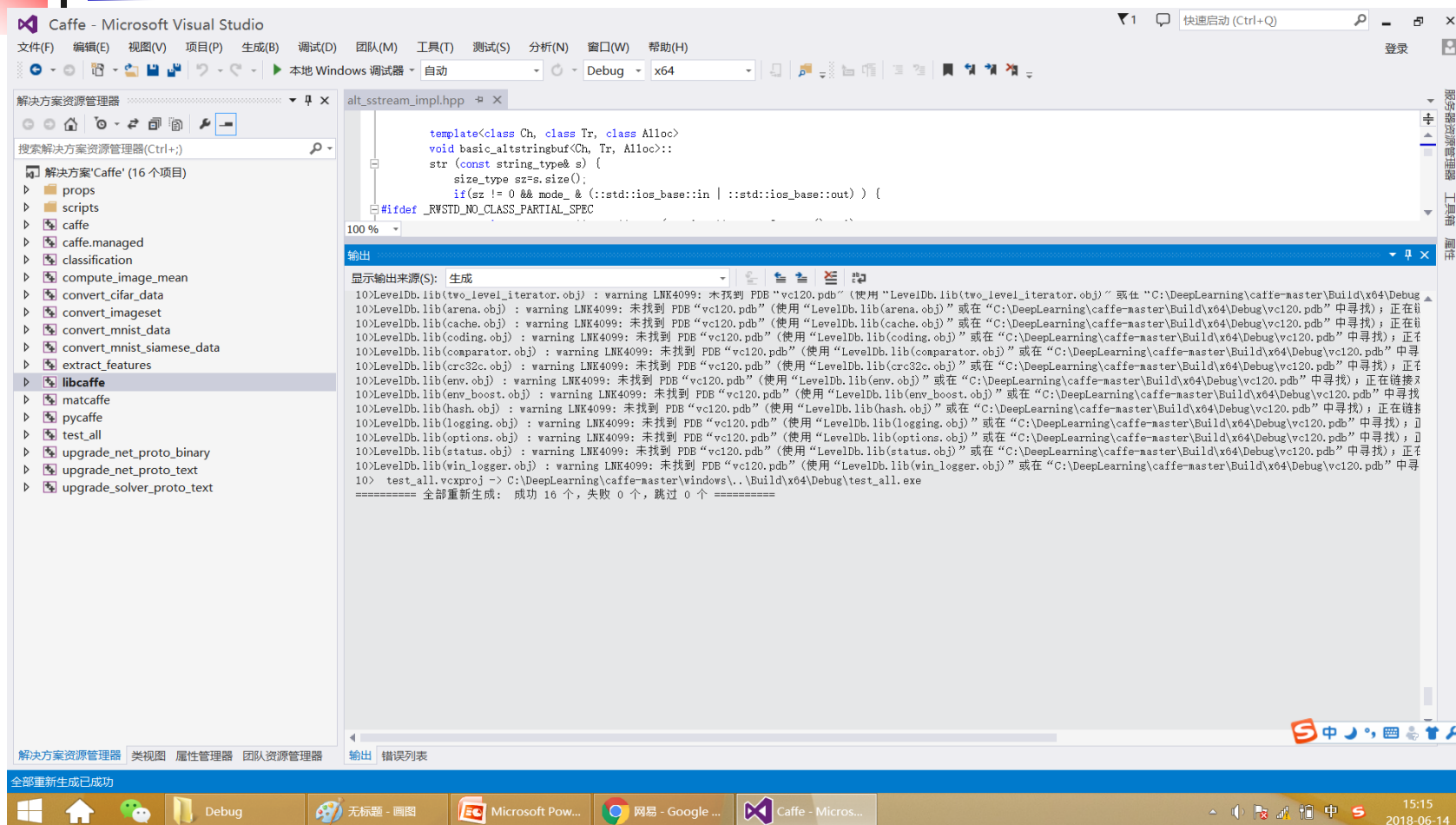
这台电脑 > Windows8_OS (C:) > DeepLearning

名称
caffe-master
LMDB
NugetPackages
caffe-master.zip

这台电脑 > Windows8_OS (C:) > DeepLearning > NugetPackages

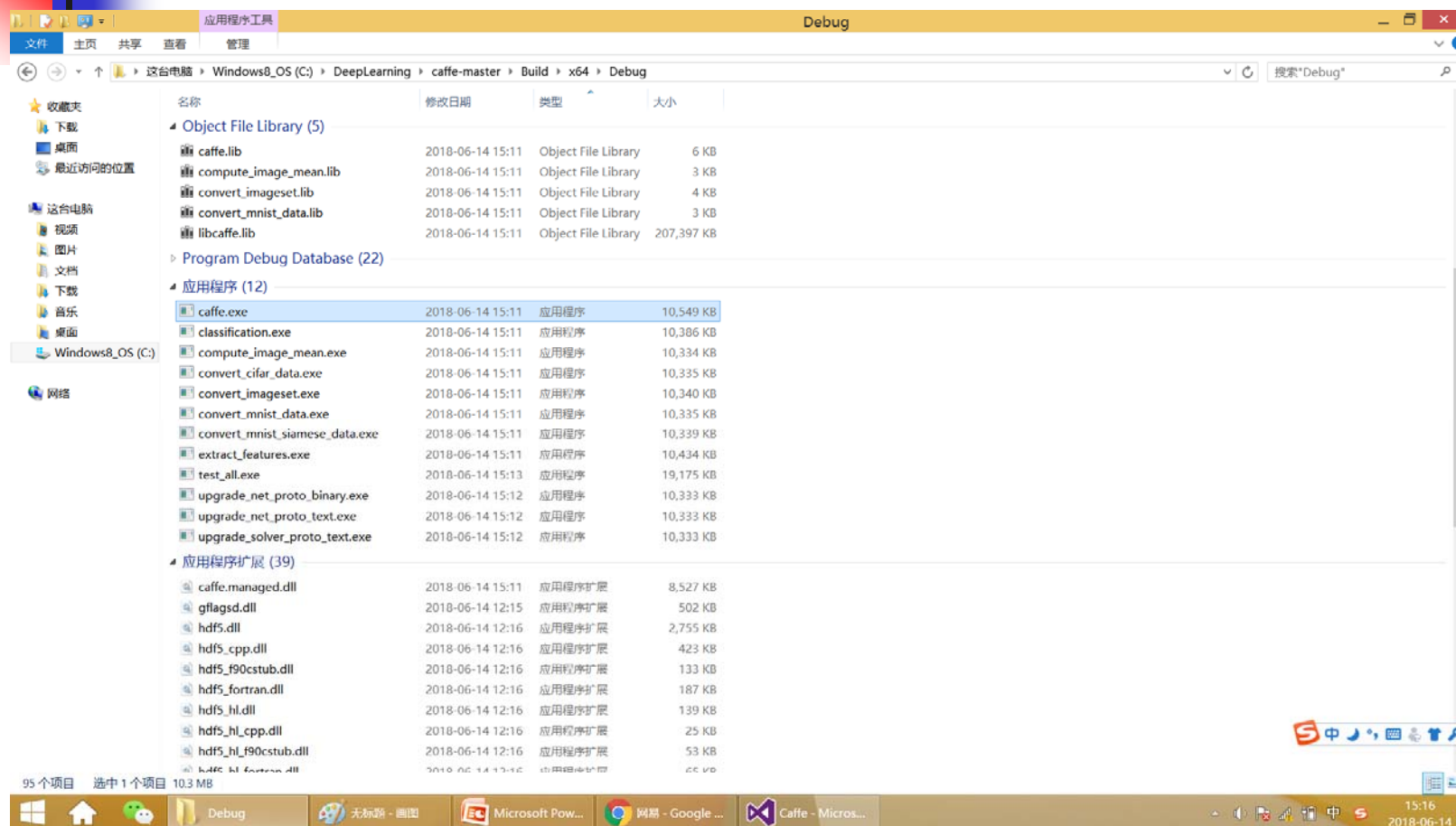
名称	修改日期	类型	大小
boost.1.59.0.0	2018-06-14 12:11	文件夹	
boost_chrono-vc120.1.59.0.0	2018-06-14 12:21	文件夹	
boost_date_time-vc120.1.59.0.0	2018-06-14 12:12	文件夹	
boost_filesystem-vc120.1.59.0.0	2018-06-14 12:21	文件夹	
boost_python2.7-vc120.1.59.0.0	2018-06-14 12:21	文件夹	
boost_system-vc120.1.59.0.0	2018-06-14 12:21	文件夹	
boost_thread-vc120.1.59.0.0	2018-06-14 12:21	文件夹	
gflags.2.1.2.1	2018-06-14 12:15	文件夹	
glog.0.3.3.0	2018-06-14 15:27	文件夹	
hdf5-v120-complete.1.8.15.2	2018-06-14 12:16	文件夹	
LevelDB-vc120.1.2.0.0	2018-06-14 12:17	文件夹	
lmdb-v120-clean.0.9.14.0	2018-06-14 12:17	文件夹	
OpenBLAS.0.2.14.1	2018-06-14 12:17	文件夹	
OpenCV.2.4.10	2018-06-14 15:27	文件夹	
protobuf-v120.2.6.1	2018-06-14 12:20	文件夹	
protoc_x64.2.6.1	2018-06-14 12:20	文件夹	

● Caffe简介与安装



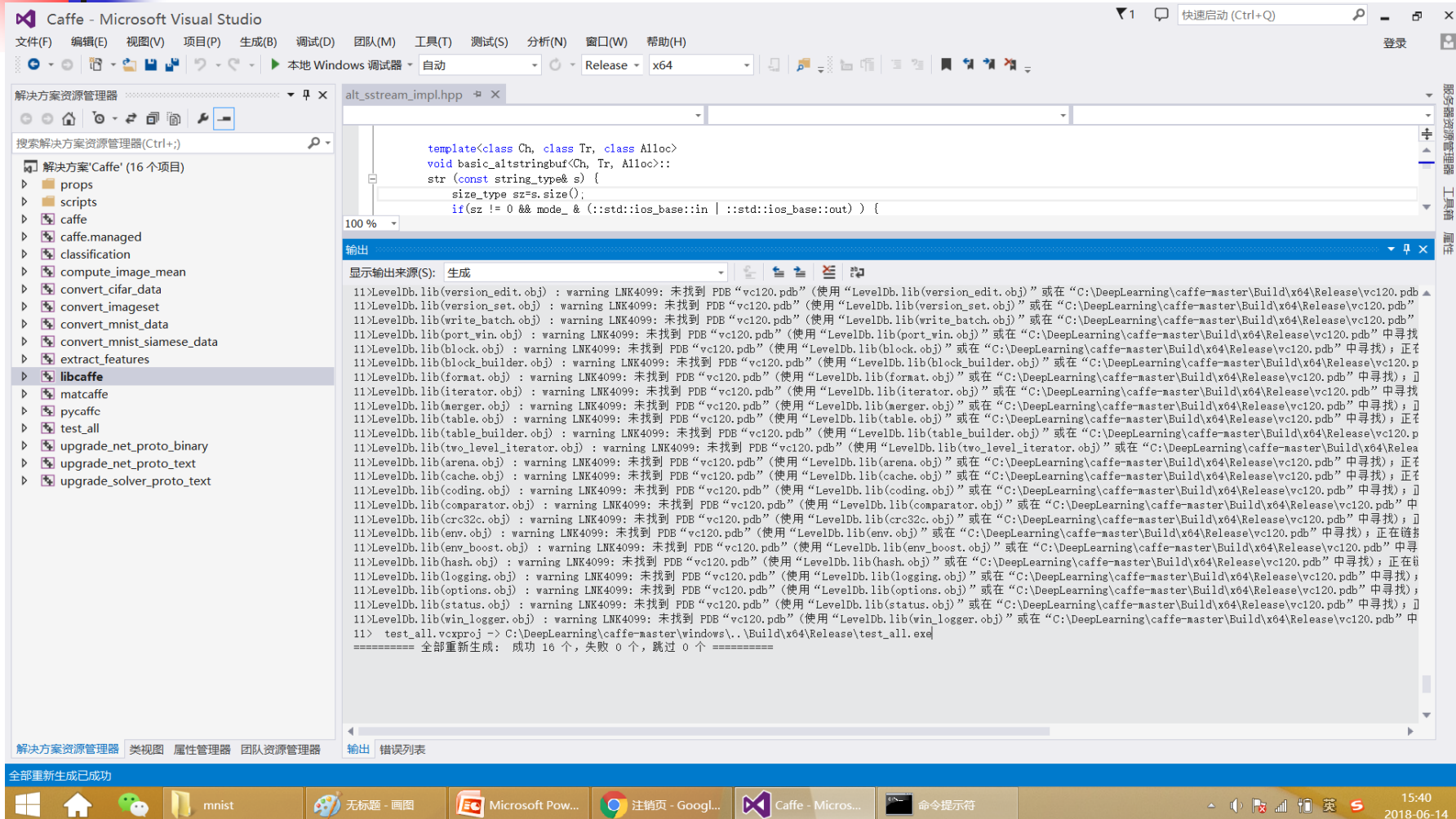
Debug版编译成功

● Caffe简介与安装



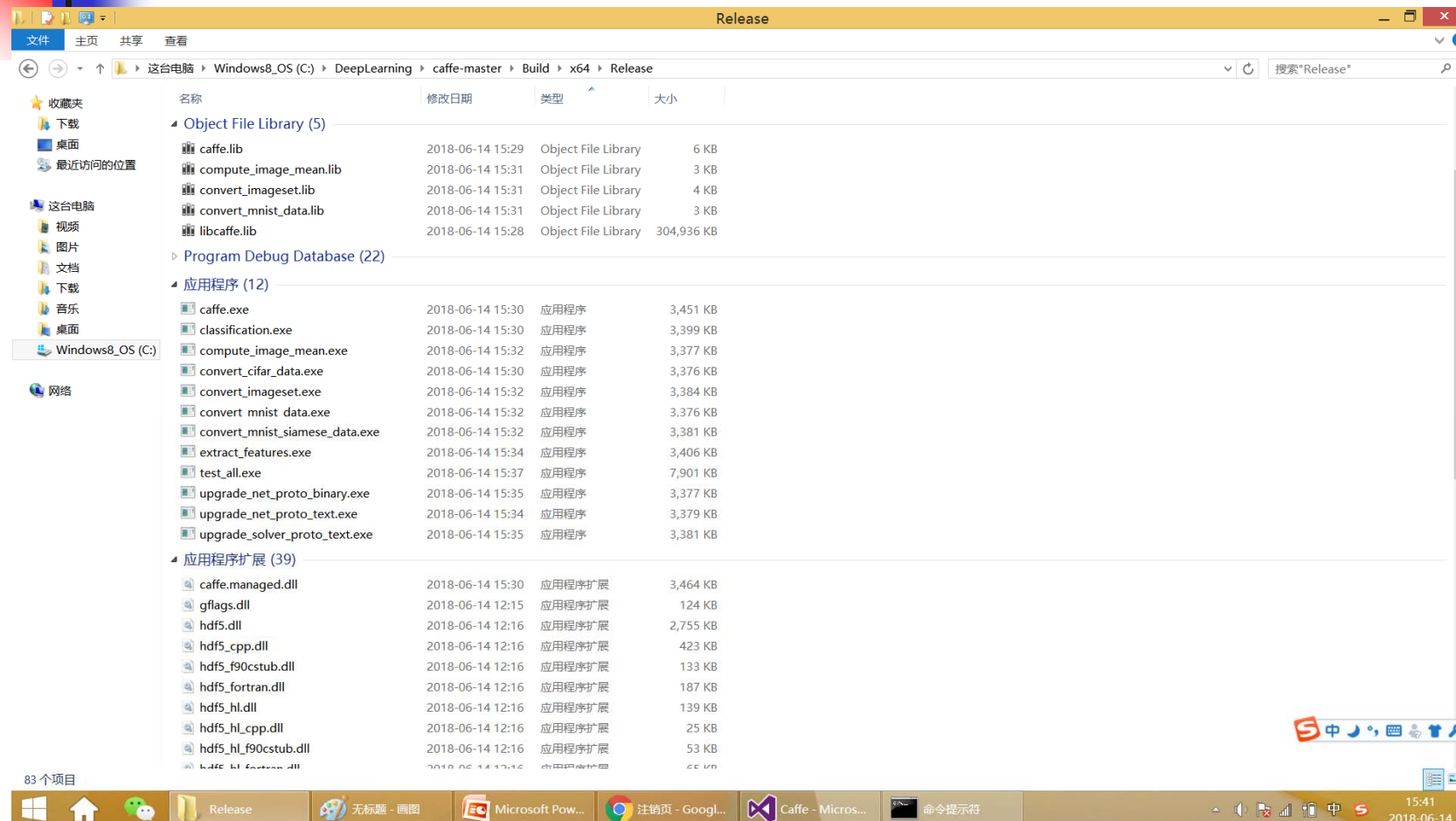
对应Debug版的库文件和应用程序

● Caffe简介与安装



Release版编译成功

● Caffe简介与安装

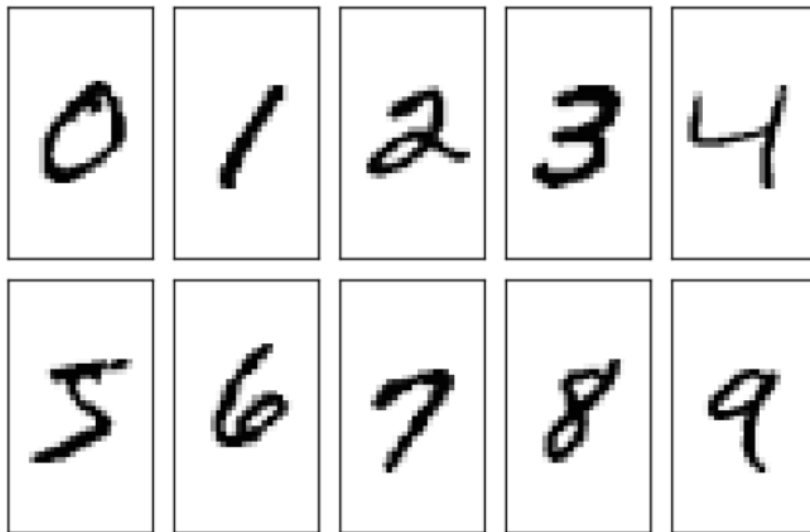


对应Release版的库文件和应用程序

- Caffe简介与安装
- 手写字识别深度模型训练与测试
- 结合OpenCV的手写字识别系统

- 手写字识别深度模型训练与测试

关于手写字MNIST数据集



MNIST(Mixed National Institute of Standards and Technology)是由纽约大学Yann LeCun教授整理

由250 个不同人手写的数字构成, 其中 50% 是高中学生, 50% 来自人口普查局工作人员。

训练集: 60000张

测试集: 10000张

尺寸: 28*28 (已归一化)

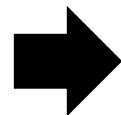
- 手写字识别深度模型训练与测试

下载MNIST数据集

方式1: <http://yann.lecun.com/exdb/mnist/>

方式2: MNIST数据集可以在Caffe源码框架的data/mnist下用get_mnist.sh脚本下载。

```
$ cd data/mnist/  
$ ./get_mnist.sh  
$ tree
```



```
get_mnist.sh  
t10k-images-idx3-ubyte.gz  
t10k-labels-idx1-ubyte.gz  
train-images-idx3-ubyte.gz  
train-labels-idx1-ubyte.gz
```

● 手写字识别深度模型训练与测试

C语言版的可视化

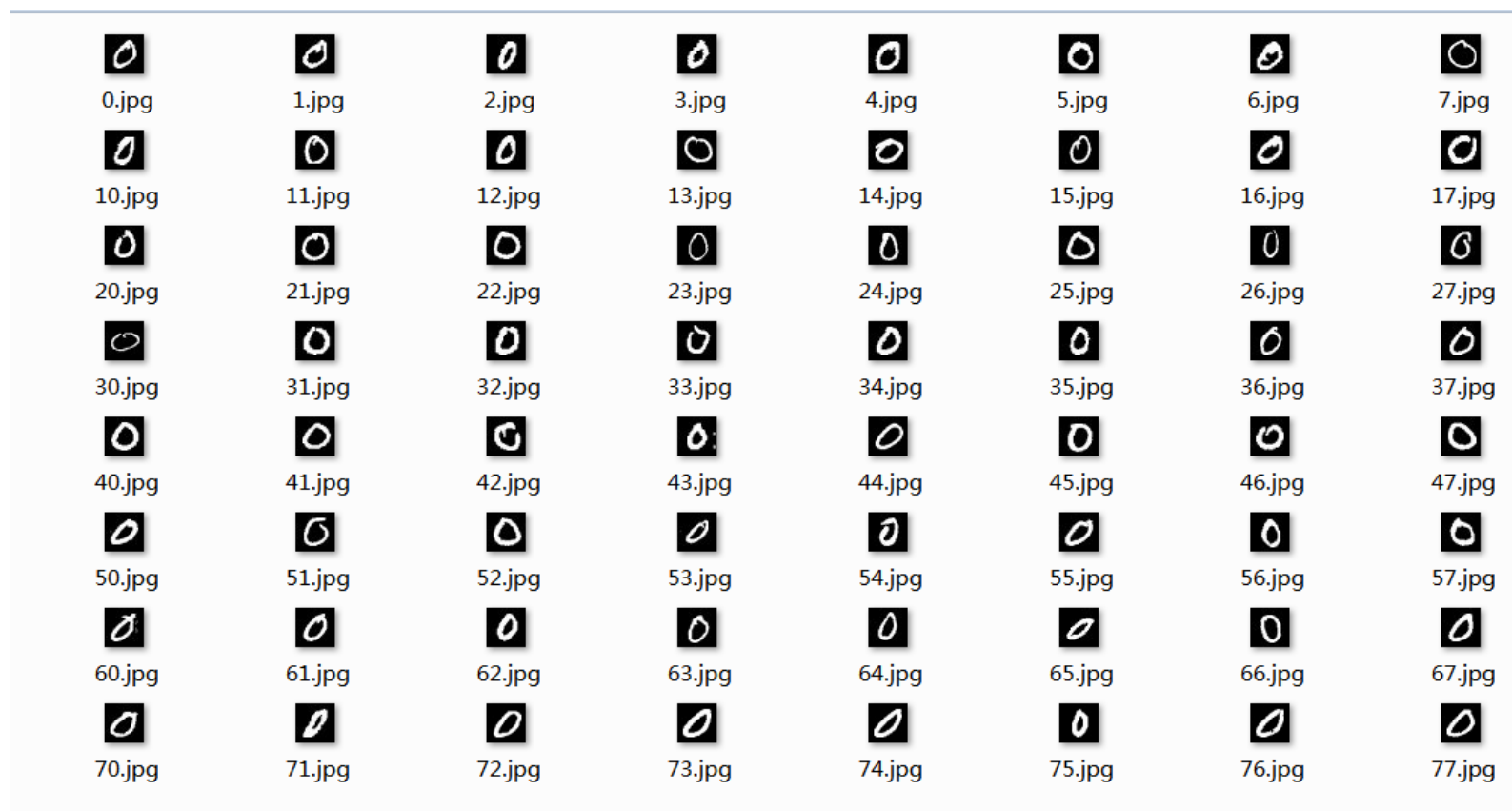
```
int _tmain(int argc, _TCHAR* argv[])
{
    string im_format = ".jpg";
    //training
    string root_name = "training";
    string image_filename = "..\\..\\train-images-idx3-ubyte";
    string label_filename = "..\\..\\train-labels-idx1-ubyte";
    //testing
    /*string root_name = "testing";
    string image_filename = "..\\..\\t10k-images-idx3-ubyte";
    string label_filename = "..\\..\\t10k-labels-idx1-ubyte";*/
    int image_size = 28 * 28;
    vector<cv::Mat> vec_images;
    read_Mnist_Image(image_filename, vec_images);
    //cout<<vec_images.size()<<endl;
    printf("image_size=%d\\n",vec_images.size());
    imshow("1st", vec_images[0]);
    vector<double> vec_labels(vec_images.size(), -1);
    read_Mnist_Label(label_filename, vec_labels);
    printf("%f\\n", vec_labels[0]);
    cvWaitKey(0);
    map<int, vector<int>> label_indexs;
    for (int i = 0; i < vec_images.size(); ++i)
    {
        int label = vec_labels[i];
        label_indexs[label].push_back(i);
    }
}
```

● 手写字识别深度模型训练与测试

```
for (map<int,vector<int>>::const_iterator ite = label_indexs.begin(); ite != label_indexs.end(); ++ite)
{
    ostringstream digit_folder;
    digit_folder << ite->first;
    string folder_name = "..\\..\\";
    folder_name = folder_name + root_name + "\\\" + digit_folder.str() + "\\\";
    mkdir(folder_name.c_str());
    printf("类别=%s\\n",folder_name.c_str());
    printf("每类大小=%d\\n",ite->second.size());

    for (int j = 0; j < ite->second.size(); ++j)
    {
        int image_idx = ite->second[j];
        ostringstream ogg;
        ogg << j;
        /*printf("folder_name=%s\\n",folder_name.c_str());
        printf("ogg=%s\\n",ogg.str().c_str());
        printf("format=%s\\n",im_format.c_str());*/
        string im_name = folder_name + ogg.str() + im_format;
        /*printf("%s\\n",im_name.c_str());*/
        imwrite(im_name, vec_images[image_idx]);
        /*printf("hereX\\n");*/
    }
}
return 0;
```

● 手写字识别深度模型训练与测试



- 手写字识别深度模型训练与测试

get_mnist.sh脚本内容

```
# !/usr/bin/env sh
# 该脚本用于下载MNIST数据集并解压

DIR="$( cd "$(dirname "$0")" ; pwd - P )"
cd #DIR

Echo "Downloading..."

Wget --no-check-certificate http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Wget --no-check-certificate http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
Wget --no-check-certificate http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
Wget --no-check-certificate http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz

Echo "Unzipping..."
train-images-idx3-ubyte.gz
train-labels-idx1-ubyte.gz
t10k-images-idx3-ubyte.gz
t10k-labels-idx1-ubyte.gz

# Creation is split out because leveldb sometimes causes segfault
# and needs to be re-created

Echo "Done."
```

● 手写字识别深度模型训练与测试

MNIST格式描述

train-images-idx3-ubyte			
文件偏移量	数据类型	值	描述
0000	32位整型	2051	(魔数) 大端存储
0004	32位整型	60000	文件包含条目总数
0008	32位整型	28	行数
0012	32位整型	28	列数
0016	8位字节	?	像素值 (0-255)
0017	8位字节	?	像素值 (0-255)
...	...	...	...

训练集图片文件格式描述

t10k-images-idx3-ubyte			
文件偏移量	数据类型	值	描述
0000	32位整型	2051	(魔数) 大端存储
0004	32位整型	10000	文件包含条目总数
0008	32位整型	28	行数
0012	32位整型	28	列数
0016	8位字节	?	像素值 (0-255)
0017	8位字节	?	像素值 (0-255)
...	...	...	...

测试集图片文件格式描述

图片文件中像素按行组织，像素值**0**表示背景（白色），
像素值**255**表示前景（黑色）

train-labels-idx1-ubyte			
文件偏移量	数据类型	值	描述
0000	32位整型	2049	(魔数) 大端存储
0004	32位整型	60000	文件包含条目总数
0008	8位字节	?	标签值 (0-9)
0009	8位字节	?	标签值 (0-9)
...	...	...	...

训练集标签文件格式描述

t10k-labels-idx1-ubyte			
文件偏移量	数据类型	值	描述
0000	32位整型	2049	(魔数) 大端存储
0004	32位整型	10000	文件包含条目总数
0008	8位字节	?	标签值 (0-9)
0009	8位字节	?	标签值 (0-9)
...	...	...	...

测试集标签文件格式描述

- 手写字识别深度模型训练与测试

MNIST格式转换

下载的原始数据集为二进制文件，需要转换为LEVELDB或LMDB才能被Caffe识别。编译好Caffe后，只需在Caffe根目录下执行以下脚本：

```
$ ./examples/mnist/create_mnist.sh
```

```
Creating Imdb...
```

```
Done.
```

目录examples/mnist下生成examples/mnist/mnist_train_lmdb/和examples/mnist/mnist_test_lmdb/两个目录，每个目录下都有两个文件：data.mdb和lock.mdb

```
$ ls -l examples/mnist/mnist_train_lmdb/
```

```
Total 60320
```

```
-rw-rw-r-- 1 yourname yourname 61763584 Oct 16 01:45 data.mdb
```

```
-rw-rw-r-- 1 yourname yourname      8192 Oct 16 01:45 lock.mdb
```

```
$ ls -l examples/mnist/mnist_test_lmdb/
```

```
Total 10100
```

```
-rw-rw-r-- 1 yourname yourname 10338304 Oct 16 01:45 data.mdb
```

```
-rw-rw-r-- 1 yourname yourname      8192 Oct 16 01:45 lock.mdb
```


- 手写字识别深度模型训练与测试

MNIST格式转换

Caffe为什么采用LMDB、LEVELDB，而不是直接读取原始数据？

- 数据类型多种多样（有二进制文件、文本文件、编码后的图像文件如JPEG或PNG、网络爬取的数据等），不可能用一套代码实现所有类型的输入数据读取，转换为统一的格式可以简化数据读取层的实现；
- 使用LMDB、LEVELDB可以提高磁盘IO利用率。

● 手写字识别深度模型训练与测试

下载MNIST数据集

<http://yann.lecun.com/exdb/mnist/>



 train-images-idx3-ubyte.gz	2018/5/24 15:33	WinRAR 压缩文件	9,681 KB
 train-labels-idx1-ubyte.gz	2018/5/24 15:33	WinRAR 压缩文件	29 KB
 t10k-images-idx3-ubyte.gz	2018/5/24 15:34	WinRAR 压缩文件	1,611 KB
 t10k-labels-idx1-ubyte.gz	2018/5/24 15:34	WinRAR 压缩文件	5 KB

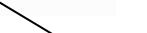
在Linux系统中将该数据转换为LMDB格式



mnist_test_lmdb
mnist_train_lmdb



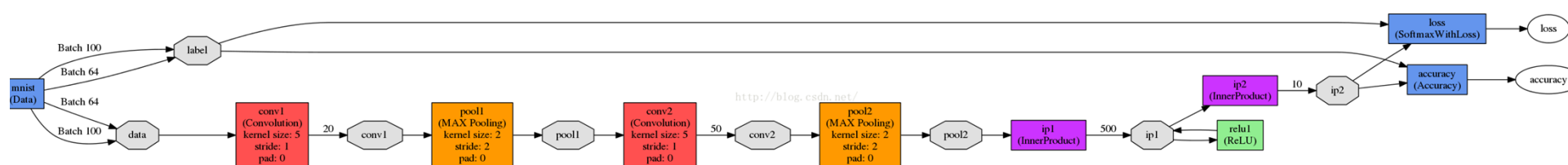
名称	修改日期	类型	大小
 data.mdb	2018/6/13 20:28	MDB 文件	10,096 KB
 lock.mdb	2018/6/13 20:28	MDB 文件	8 KB



名称	修改日期	类型	大小
 data.mdb	2018/6/13 20:30	MDB 文件	60,316 KB
 lock.mdb	2018/6/13 20:30	MDB 文件	8 KB

● 手写字识别深度模型训练与测试

认识LeNet-5深度网络模型 （描述文件：examples/mnist/lenet_train_val.prototxt）



● 手写字识别深度模型训练与测试

认识LeNet-5深度网络模型

```
Name: "LeNet"           //网络(Net)的名称为LeNet
Layer{                  //定义一个层(Layer)
  name: "mnist"          //层的名称为mnist
  type: "Data"           //层的类型为数据层
  top: "data"            //层的输出blob有两个: data和label
  top: "label"
  include {
    phase: TRAIN         //该层参数只在训练阶段有效
  }
  transform_param {
    scale: 0.00390625    //数据变换使用的数据缩放因子
  }
  data_param {           //数据层参数
    source: "examples/mnist/mnist_train_lmdb" //LMDB的路径
    batch_size: 64        //批量数目, 一次读取64张图
    backend: LMDB          //数据格式为LMDB
  }
}
```

● 手写字识别深度模型训练与测试

认识LeNet-5深度网络模型

```
Layer{                                     //一个新的数据层，名字也是mnist，输出blob也是data和label，但
是这里定义参数只在分类阶段有效
  name: "mnist"                           //层的名称为mnist
  type: "Data"                           //层的类型为数据层
  top: "data"                             //层的输出blob有两个：data和label
  top: "label"
  include {
    phase: TEST                           //该层参数只在测试阶段有效
  }
  transform_param {
    scale: 0.00390625                     //数据变换使用的数据缩放因子
  }
  data_param {                             //数据层参数
    source: "examples/mnist/mnist_test_lmdb" //LMDB的路径
    batch_size: 100                       //批量数目，一次读取100张图
    backend: LMDB                         //数据格式为LMDB
  }
}
```


● 手写字识别深度模型训练与测试

```
Layer{                                     //定义一个新的卷积层conv1，输入blob为data，输出blob为conv
  name: "conv1"                           //层的名称
  type: "Convolution"                     //层的类型
  bottom: "data"
  top: "conv1"
  param {
    lr_mult: 1                           //权值学习速率倍乘因子，1倍表示保持与全局参数一致
  }
  param {
    lr_mult: 2                           //bias学习速率倍乘因子，是全局参数的2倍
  }
  convolution_param {                    //卷积计算参数
    num_output: 20                       //输出feature map数目为20
    kernel_size: 5                       //卷积核尺寸，5*5
    stride: 1                            //卷积输出跳跃间隔，1表示连续输出，无跳跃
    weight_filler {                      //权值使用xavier 填充器（一种神经网络初始化方法）
      type: "xavier"
    }
    bias_filler {                        // bias使用常数填充器，默认为0
      type: "constant"
    }
  }
}
```


- 手写字识别深度模型训练与测试

```
Layer{                                     //定义新的下采样层pool1，输入blob为conv1，输出blob为pool1
  name: "pool1"                           //层的名称
  type: "Pooling"                         //层的类型
  bottom: "conv1"
  top: "pool1"
  pooling_param {                         //下采样参数
    pool: MAX                             //使用最大值下采样方法
    kernel_size: 2                        //下采样窗口尺寸2*2
    stride: 2                             //下采样输出跳跃间隔2*2
  }
}
```

● 手写字识别深度模型训练与测试

```
Layer{                                     //定义一个新的卷积层conv2，与conv1类似
  name: "conv2"                           //层的名称
  type: "Convolution"                     //层的类型
  bottom: "pool1"
  top: "conv2"
  param {
    lr_mult: 1                           //权值学习速率倍乘因子，1倍表示保持与全局参数一致
  }
  param {
    lr_mult: 2                           //bias学习速率倍乘因子，是全局参数的2倍
  }
  convolution_param {                     //卷积计算参数
    num_output: 50                       //输出feature map数目为50
    kernel_size: 5                       //卷积核尺寸，5*5
    stride: 1                            //卷积输出跳跃间隔，1表示连续输出，无跳跃
    weight_filler {                      //权值使用xavier 填充器（一种神经网络初始化方法）
      type: "xavier"
    }
    bias_filler {                        // bias使用常数填充器，默认为0
      type: "constant"
    }
  }
}
```


- 手写字识别深度模型训练与测试

```
Layer{                                     //定义新的下采样层pool2, 与pool1类似
  name: "pool2"                           //层的名称
  type: "Pooling"                         //层的类型
  bottom: "conv2"
  top: "pool2"
  pooling_param {                         //下采样参数
    pool: MAX                             //使用最大值下采样方法
    kernel_size: 2                       //下采样窗口尺寸2*2
    stride: 2                             //下采样输出跳跃间隔2*2
  }
}
```

● 手写字识别深度模型训练与测试

```
Layer{                                     //新的全连接层，输入blob为pool2，输出blob为ip1
  name: "ip1"                             //层的名称
  type: "InnerProduct"                   //层的类型
  bottom: "pool2"
  top: "ip1"
  param {
    lr_mult: 1                           //权值学习速率倍乘因子，1倍表示保持与全局参数一致
  }
  param {
    lr_mult: 2                           //bias学习速率倍乘因子，是全局参数的2倍
  }
  inner_product_param { //全连接层参数
    num_output: 500                //该层输出元素个数为500
    weight_filler {                //权值使用xavier 填充器（一种神经网络初始化方法）
      type: "xavier"
    }
    bias_filler {                  // bias使用常数填充器，默认为0
      type: "constant"
    }
  }
}
```

● 手写字识别深度模型训练与测试

```
Layer{           //新的非线性层，用ReLU方法
  name: "relu1"   //层的名称
  type: "ReLU"    //层的类型
  bottom: "ip1"
  top: "ip1"
}
```

1.为什么引入非线性激励函数？

如果不适用激励函数，那么在这种情况下每一层的输出都是上层输入的线性函数，很容易验证，无论你神经网络有多少层，输出都是输入的线性组合，与没有隐藏层效果相当，这种情况就是最原始的感知机（perceptron）了

正因为上面的原因，我们决定引入非线性函数作为激励函数，这样深层神经网络就有意义了，不再是输入的线性组合，可以逼近任意函数，最早的想法是用sigmoid函数或者tanh函数，输出有界，很容易充当下一层的输入

2.为什么引入Relu？

第一，采用sigmoid等函数，算激活函数时候（指数运算），计算量大，反向传播求误差梯度时，求导涉及除法，计算量相当大，而采用Relu激活函数，整个过程的计算量节省很多

第二，对于深层网络，sigmoid函数反向传播时，很容易就出现梯度消失的情况（在sigmoid函数接近饱和区时，变换太缓慢，导数趋于0，这种情况会造成信息丢失），从而无法完成深层网络的训练

第三，Relu会使一部分神经元的输出为0，这样就造成了网络的稀疏性，并且减少了参数的相互依存关系，缓解了过拟合问题的发生

● 手写字识别深度模型训练与测试

```
Layer{                                     //全连接层，输入blob为ip1，输出blob为ip2
  name: "ip2"                             //层的名称
  type: "InnerProduct"                   //层的类型
  bottom: "ip1"
  top: "ip2"
  param {
    lr_mult: 1                           //权值学习速率倍乘因子，1倍表示保持与全局参数一致
  }
  param {
    lr_mult: 2                           //bias学习速率倍乘因子，是全局参数的2倍
  }
  inner_product_param { //全连接层参数
    num_output: 10                      //该层输出元素个数为10
    weight_filler {                     //权值使用xavier 填充器（一种神经网络初始化方法）
      type: "xavier"
    }
    bias_filler {                       // bias使用常数填充器，默认为0
      type: "constant"
    }
  }
}
```

● 手写字识别深度模型训练与测试

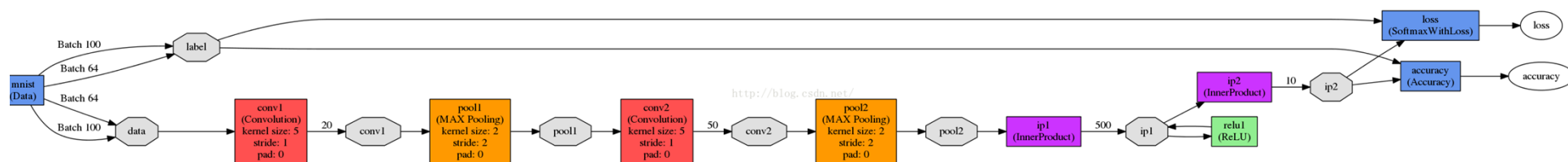
```
Layer{                                     //分类准确率层，只在Testing 阶段有效， 输入blob为ip2和label， 输出blob为accuracy
    name: "accuracy"                       //层的名称
    type: "Accuracy"                       //层的类型
    bottom: "ip2"
    bottom: "label"
    top: "accuracy"
    include {
        phase: TEST
    }
}
```


- 手写字识别深度模型训练与测试

```
Layer{                                //损失层，损失函数采用SoftmaxLoss，输入blob为ip2和label，输出
blob为loss
  name: "loss"                        //层的名称
  type: "SoftmaxWithLoss"            //层的类型
  bottom: "ip2"
  bottom: "label"
  top: "loss"
}
```

● 手写字识别深度模型训练与测试

认识LeNet-5深度网络模型



LeNet网络结构

数据源mnist: 从预处理得到的Imdb数据库中读取图像数据data和标签label, 并输入CNN进行处理

CNN包括:

- 一组由卷积层conv(1,2)+下采样层pool(1,2)交替形成的特征层
- 两个全连接层ip1和ip2（类似多层感知器结构）
- 对ip2的输出进一步同标签数据labelUI比, 计算准确率和损失

● 手写字识别深度模型训练与测试

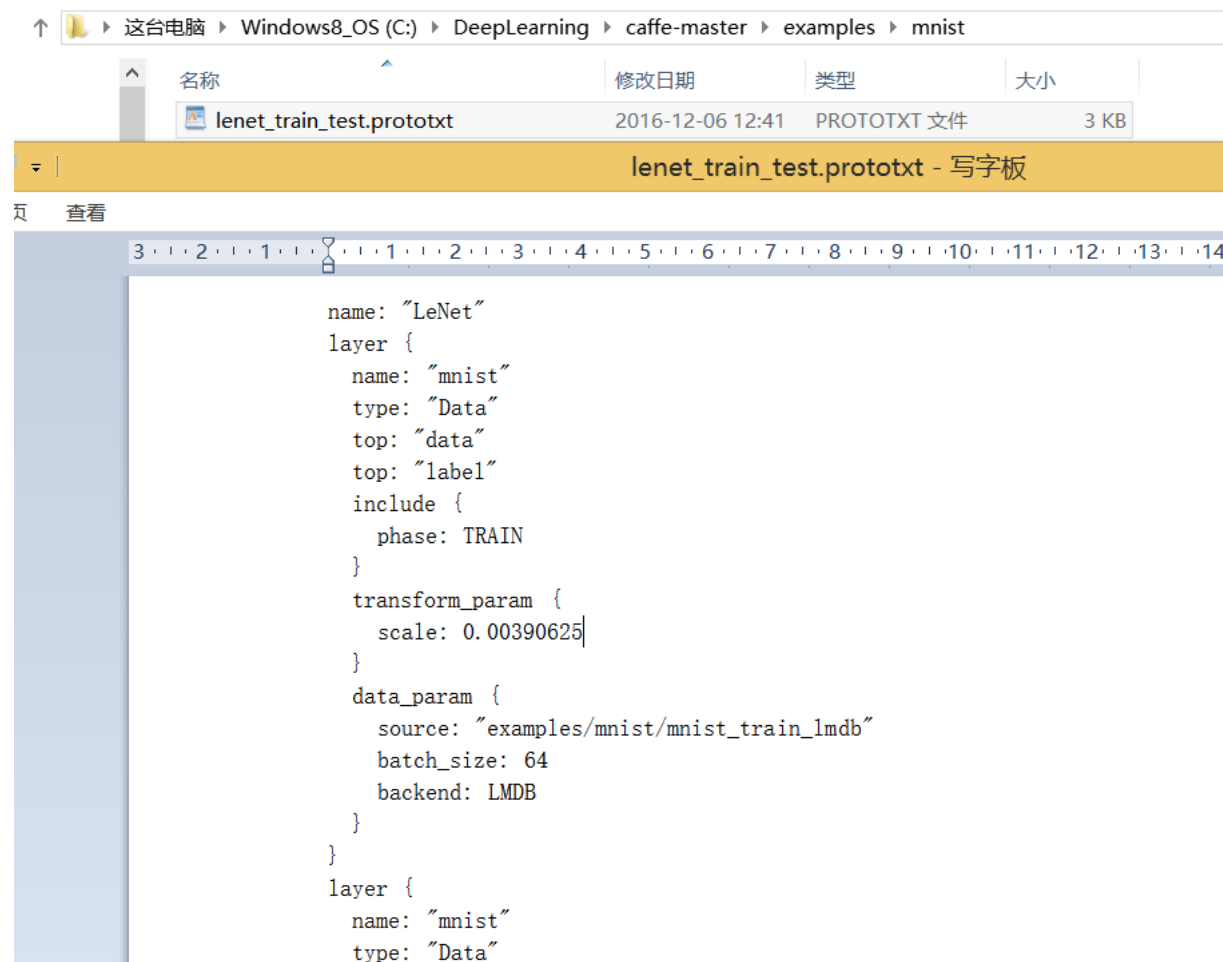
把转换好的数据
拷贝到
examples\mnist
文件夹里面

这台电脑 > Windows8_OS (C:) > DeepLearning > caffe-master > examples > mnist >

名称	修改日期	类型	大小
mnist_test_lmdb	2018-06-14 15:45	文件夹	
mnist_train_lmdb	2018-06-14 15:45	文件夹	
convert_mnist_data.cpp	2016-12-06 12:41	C++ Source	5 KB
create_mnist.sh	2016-12-06 12:41	SH 文件	1 KB
lenet.prototxt	2016-12-06 12:41	PROTOTXT 文件	2 KB
lenet_adadelta_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_auto_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_consolidated_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	6 KB
lenet_multistep_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_solver_adam.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_solver_rmsprop.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_train_test.prototxt	2016-12-06 12:41	PROTOTXT 文件	3 KB
mnist_autoencoder.prototxt	2016-12-06 12:41	PROTOTXT 文件	5 KB
mnist_autoencoder_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
mnist_autoencoder_solver_adadelta....	2016-12-06 12:41	PROTOTXT 文件	1 KB
mnist_autoencoder_solver_adagrad....	2016-12-06 12:41	PROTOTXT 文件	1 KB
mnist_autoencoder_solver_nesterov....	2016-12-06 12:41	PROTOTXT 文件	1 KB
readme.md	2016-12-06 12:41	MD 文件	12 KB
train_lenet.sh	2016-12-06 12:41	SH 文件	1 KB
train_lenet_adam.sh	2016-12-06 12:41	SH 文件	1 KB
train_lenet_consolidated.sh	2016-12-06 12:41	SH 文件	1 KB
train_lenet_docker.sh	2016-12-06 12:41	SH 文件	5 KB
train_lenet_rmsprop.sh	2016-12-06 12:41	SH 文件	1 KB
train_mnist_autoencoder.sh	2016-12-06 12:41	SH 文件	1 KB
train_mnist_autoencoder_adadelta.sh	2016-12-06 12:41	SH 文件	1 KB
train_mnist_autoencoder_adagrad.sh	2016-12-06 12:41	SH 文件	1 KB
train_mnist_autoencoder_nesterov.sh	2016-12-06 12:41	SH 文件	1 KB

● 手写字识别深度模型训练与测试

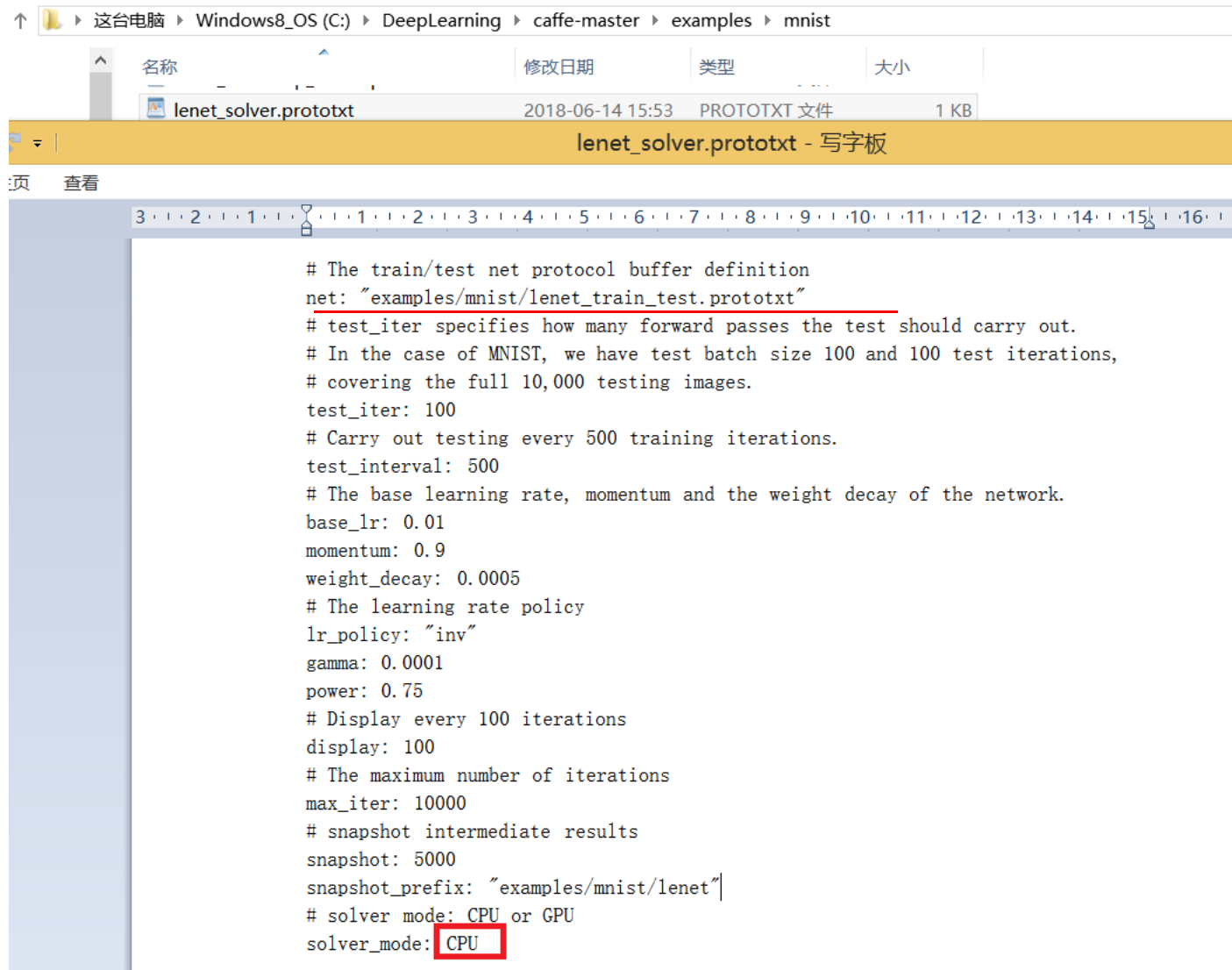
确定使用的
LeNet
模型



```
name: "LeNet"
layer {
  name: "mnist"
  type: "Data"
  top: "data"
  top: "label"
  include {
    phase: TRAIN
  }
  transform_param {
    scale: 0.00390625
  }
  data_param {
    source: "examples/mnist/mnist_train_lmdb"
    batch_size: 64
    backend: LMDB
  }
}
layer {
  name: "mnist"
  type: "Data"
```

● 手写字识别深度模型训练与测试

确定求解
模式和参
数



```
# The train/test net protocol buffer definition
net: "examples/mnist/lenet_train_test.prototxt"
# test_iter specifies how many forward passes the test should carry out.
# In the case of MNIST, we have test batch size 100 and 100 test iterations,
# covering the full 10,000 testing images.
test_iter: 100
# Carry out testing every 500 training iterations.
test_interval: 500
# The base learning rate, momentum and the weight decay of the network.
base_lr: 0.01
momentum: 0.9
weight_decay: 0.0005
# The learning rate policy
lr_policy: "inv"
gamma: 0.0001
power: 0.75
# Display every 100 iterations
display: 100
# The maximum number of iterations
max_iter: 10000
# snapshot intermediate results
snapshot: 5000
snapshot_prefix: "examples/mnist/lenet"
# solver mode: CPU or GPU
solver_mode: CPU
```

- 手写字识别深度模型训练与测试



```
命令提示符
C:\DeepLearning\caffe-master>Build\x64\Release\caffe.exe train -solver examples\mnist\lenet_solver.prototxt_
```

注意：Build
文件夹和
examples文
件夹都在
caffe-
master文件
夹下面

在命令窗口进入caffe-master目录，然后输入命令：

C:\DeepLearning\caffe-master> Build\x64\Release\caffe.exe
train -solver examples\mnist\lenet_solver.prototxt

● 手写字识别深度模型训练与测试

```
命令提示符 - Build\x64\Release\caffe.exe train -solver examples\mnist\... -
I0614 16:01:40.645434 8528 net.cpp:228] label_mnist_1_split does not need backward computation.
I0614 16:01:40.645434 8528 net.cpp:228] mnist does not need backward computation.
I0614 16:01:40.645434 8528 net.cpp:270] This network produces output accuracy
I0614 16:01:40.645434 8528 net.cpp:270] This network produces output loss
I0614 16:01:40.645434 8528 net.cpp:283] Network initialization done.
I0614 16:01:40.645434 8528 solver.cpp:60] Solver scaffolding done.
I0614 16:01:40.645434 8528 caffe.cpp:252] Starting Optimization
I0614 16:01:40.645434 8528 solver.cpp:279] Solving LeNet
I0614 16:01:40.645434 8528 solver.cpp:280] Learning Rate Policy: inv
I0614 16:01:40.645434 8528 solver.cpp:337] Iteration 0, Testing net (#0)
I0614 16:01:43.706562 8528 solver.cpp:404] Test net output #0: accuracy = 0.046
I0614 16:01:43.706562 8528 solver.cpp:404] Test net output #1: loss = 2.40965 (* 1 = 2.40965 loss)
I0614 16:01:43.769062 8528 solver.cpp:228] Iteration 0, loss = 2.39801
I0614 16:01:43.769062 8528 solver.cpp:244] Train net output #0: loss = 2.39801 (* 1 = 2.39801 loss)
I0614 16:01:43.769062 8528 sgd_solver.cpp:106] Iteration 0, lr = 0.01
I0614 16:01:48.363593 8528 solver.cpp:228] Iteration 100, loss = 0.21502
I0614 16:01:48.363593 8528 solver.cpp:244] Train net output #0: loss = 0.21502 (* 1 = 0.21502 loss)
I0614 16:01:48.363593 8528 sgd_solver.cpp:106] Iteration 100, lr = 0.00992565
```

训练过程

● 手写字识别深度模型训练与测试

```
命令提示符

I0614 16:10:39.792457 8528 solver.cpp:244] Train net output #0: loss = 0.00317973 (* 1 = 0.00317973 loss)
I0614 16:10:39.792457 8528 sgd_solver.cpp:106] Iteration 9700, lr = 0.00601382
I0614 16:10:44.673946 8528 solver.cpp:228] Iteration 9800, loss = 0.0141003
I0614 16:10:44.673946 8528 solver.cpp:244] Train net output #0: loss = 0.0141001 (* 1 = 0.0141001 loss)
I0614 16:10:44.673946 8528 sgd_solver.cpp:106] Iteration 9800, lr = 0.00599102
I0614 16:10:49.919406 8528 solver.cpp:228] Iteration 9900, loss = 0.00637386
I0614 16:10:49.935034 8528 solver.cpp:244] Train net output #0: loss = 0.00637368 (* 1 = 0.00637368 loss)
I0614 16:10:49.935034 8528 sgd_solver.cpp:106] Iteration 9900, lr = 0.00596843
I0614 16:10:58.225000 8528 solver.cpp:454] Snapshotting to binary proto file examples/mnist/lenet_iter_10000.caffemodel
I0614 16:10:58.540199 8528 sgd_solver.cpp:273] Snapshotting solver state to binary proto file examples/mnist/lenet_iter_10000.solverstate
I0614 16:10:58.606709 8528 solver.cpp:317] Iteration 10000, loss = 0.0028068
I0614 16:10:58.606709 8528 solver.cpp:337] Iteration 10000, Testing net (#0)
I0614 16:11:02.530758 8528 solver.cpp:404] Test net output #0: accuracy = 0.99
I0614 16:11:02.552603 8528 solver.cpp:404] Test net output #1: loss = 0.0295659 (* 1 = 0.0295659 loss)
I0614 16:11:02.552603 8528 solver.cpp:322] Optimization Done.
I0614 16:11:02.553607 8528 caffe.cpp:255] Optimization Done.

C:\DeepLearning\caffe-master>
```

训练结束

● 手写字识别深度模型训练与测试

这台电脑 ▸ Windows8_OS (C:) ▸ DeepLearning ▸ caffe-master ▸ examples ▸ mnist

名称	修改日期	类型	大小
mnist_test_lmdb	2018-06-14 15:45	文件夹	
mnist_train_lmdb	2018-06-14 15:45	文件夹	
convert_mnist_data.cpp	2016-12-06 12:41	C++ Source	5 KB
create_mnist.sh	2016-12-06 12:41	SH 文件	1 KB
lenet.prototxt	2016-12-06 12:41	PROTOTXT 文件	2 KB
lenet_adadelta_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_auto_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_consolidated_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	6 KB
lenet_iter_5000.caffemodel	2018-06-14 16:06	CAFFEMODEL 文件	1,685 KB
lenet_iter_5000.solverstate	2018-06-14 16:06	SOLVERSTATE ...	1,685 KB
lenet_iter_10000.caffemodel	2018-06-14 16:10	CAFFEMODEL 文件	1,685 KB
lenet_iter_10000.solverstate	2018-06-14 16:10	SOLVERSTATE ...	1,685 KB
lenet_multistep_solver.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_solver.prototxt	2018-06-14 16:05	PROTOTXT 文件	1 KB
lenet_solver_adam.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_solver_rmsprop.prototxt	2016-12-06 12:41	PROTOTXT 文件	1 KB
lenet_train_test.prototxt	2016-12-06 12:41	PROTOTXT 文件	3 KB
mnist_autoencoder.prototxt	2016-12-06 12:41	PROTOTXT 文件	5 KB

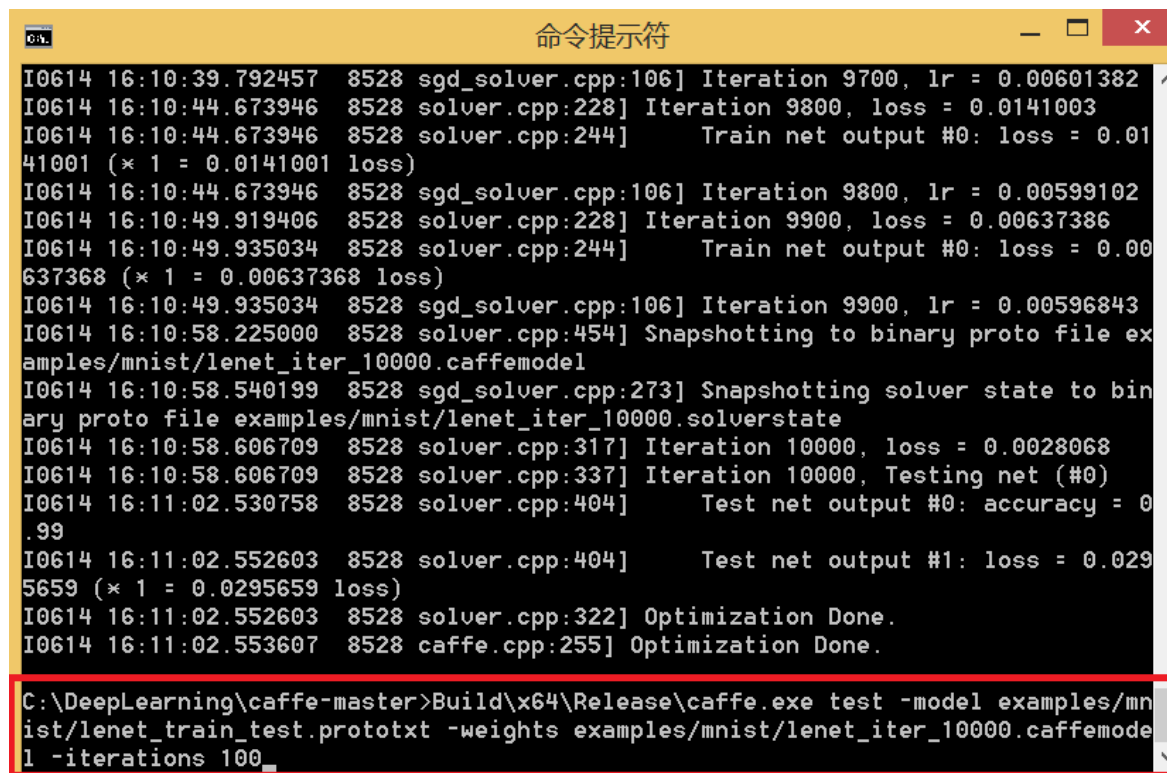
最终训练的模型权值保存在examples/mnist/lenet_iter_10000.caffemodel中

训练状态保存在examples/mnist/lenet_iter_10000.solverstate文件中

● 手写字识别深度模型训练与测试

利用训练好的LeNet-5 模型权值文件进行预测。运行命令如下：

```
C:\DeepLearning\caffe-master>Build\x64\Release\caffe.exe test -model  
examples/mnist/lenet_train_test.prototxt -weights  
examples/mnist/lenet_iter_10000.caffemodel -iterations 100
```



```
命令提示符
I0614 16:10:39.792457 8528 sgd_solver.cpp:106] Iteration 9700, lr = 0.00601382
I0614 16:10:44.673946 8528 solver.cpp:228] Iteration 9800, loss = 0.0141003
I0614 16:10:44.673946 8528 solver.cpp:244] Train net output #0: loss = 0.0141001 (* 1 = 0.0141001 loss)
I0614 16:10:44.673946 8528 sgd_solver.cpp:106] Iteration 9800, lr = 0.00599102
I0614 16:10:49.919406 8528 solver.cpp:228] Iteration 9900, loss = 0.00637386
I0614 16:10:49.935034 8528 solver.cpp:244] Train net output #0: loss = 0.00637368 (* 1 = 0.00637368 loss)
I0614 16:10:49.935034 8528 sgd_solver.cpp:106] Iteration 9900, lr = 0.00596843
I0614 16:10:58.225000 8528 solver.cpp:454] Snapshotting to binary proto file examples/mnist/lenet_iter_10000.caffemodel
I0614 16:10:58.540199 8528 sgd_solver.cpp:273] Snapshotting solver state to binary proto file examples/mnist/lenet_iter_10000.solverstate
I0614 16:10:58.606709 8528 solver.cpp:317] Iteration 10000, loss = 0.0028068
I0614 16:10:58.606709 8528 solver.cpp:337] Iteration 10000, Testing net (#0)
I0614 16:11:02.530758 8528 solver.cpp:404] Test net output #0: accuracy = 0.99
I0614 16:11:02.552603 8528 solver.cpp:404] Test net output #1: loss = 0.0295659 (* 1 = 0.0295659 loss)
I0614 16:11:02.552603 8528 solver.cpp:322] Optimization Done.
I0614 16:11:02.553607 8528 caffe.cpp:255] Optimization Done.

C:\DeepLearning\caffe-master>Build\x64\Release\caffe.exe test -model examples/mnist/lenet_train_test.prototxt -weights examples/mnist/lenet_iter_10000.caffemodel -iterations 100
```

● 手写字识别深度模型训练与测试

```
命令提示符 - Build\x64\Release\caffe.exe test -model examples/mnist/...  
I0614 16:42:53.832574 8992 caffe.cpp:309] Batch 84, accuracy = 0.99  
I0614 16:42:53.832574 8992 caffe.cpp:309] Batch 84, loss = 0.0197189  
I0614 16:42:53.863824 8992 caffe.cpp:309] Batch 85, accuracy = 1  
I0614 16:42:53.863824 8992 caffe.cpp:309] Batch 85, loss = 0.0083689  
I0614 16:42:53.895074 8992 caffe.cpp:309] Batch 86, accuracy = 1  
I0614 16:42:53.895074 8992 caffe.cpp:309] Batch 86, loss = 5.31039e-005  
I0614 16:42:53.926324 8992 caffe.cpp:309] Batch 87, accuracy = 1  
I0614 16:42:53.926324 8992 caffe.cpp:309] Batch 87, loss = 8.92651e-005  
I0614 16:42:53.957574 8992 caffe.cpp:309] Batch 88, accuracy = 1  
I0614 16:42:53.957574 8992 caffe.cpp:309] Batch 88, loss = 2.44098e-005  
I0614 16:42:53.988826 8992 caffe.cpp:309] Batch 89, accuracy = 1  
I0614 16:42:53.988826 8992 caffe.cpp:309] Batch 89, loss = 2.7017e-005  
I0614 16:42:54.020196 8992 caffe.cpp:309] Batch 90, accuracy = 0.97  
I0614 16:42:54.020196 8992 caffe.cpp:309] Batch 90, loss = 0.0472327  
I0614 16:42:54.051450 8992 caffe.cpp:309] Batch 91, accuracy = 1  
I0614 16:42:54.051450 8992 caffe.cpp:309] Batch 91, loss = 5.01804e-005  
I0614 16:42:54.082698 8992 caffe.cpp:309] Batch 92, accuracy = 1  
I0614 16:42:54.082698 8992 caffe.cpp:309] Batch 92, loss = 0.000118642  
I0614 16:42:54.113950 8992 caffe.cpp:309] Batch 93, accuracy = 1  
I0614 16:42:54.113950 8992 caffe.cpp:309] Batch 93, loss = 0.00054272  
I0614 16:42:54.145200 8992 caffe.cpp:309] Batch 94, accuracy = 1  
I0614 16:42:54.145200 8992 caffe.cpp:309] Batch 94, loss = 0.000514186  
I0614 16:42:54.176450 8992 caffe.cpp:309] Batch 95, accuracy = 1  
I0614 16:42:54.176450 8992 caffe.cpp:309] Batch 95, loss = 0.00256424
```

预测过程

- 手写字识别深度模型训练与测试

```
命令提示符
I0614 16:42:54.020196 8992 caffe.cpp:309] Batch 90, loss = 0.0472327
I0614 16:42:54.051450 8992 caffe.cpp:309] Batch 91, accuracy = 1
I0614 16:42:54.051450 8992 caffe.cpp:309] Batch 91, loss = 5.01804e-005
I0614 16:42:54.082698 8992 caffe.cpp:309] Batch 92, accuracy = 1
I0614 16:42:54.082698 8992 caffe.cpp:309] Batch 92, loss = 0.000118642
I0614 16:42:54.113950 8992 caffe.cpp:309] Batch 93, accuracy = 1
I0614 16:42:54.113950 8992 caffe.cpp:309] Batch 93, loss = 0.00054272
I0614 16:42:54.145200 8992 caffe.cpp:309] Batch 94, accuracy = 1
I0614 16:42:54.145200 8992 caffe.cpp:309] Batch 94, loss = 0.000514186
I0614 16:42:54.176450 8992 caffe.cpp:309] Batch 95, accuracy = 1
I0614 16:42:54.176450 8992 caffe.cpp:309] Batch 95, loss = 0.00256424
I0614 16:42:54.207700 8992 caffe.cpp:309] Batch 96, accuracy = 0.97
I0614 16:42:54.207700 8992 caffe.cpp:309] Batch 96, loss = 0.0678068
I0614 16:42:54.238958 8992 caffe.cpp:309] Batch 97, accuracy = 0.97
I0614 16:42:54.238958 8992 caffe.cpp:309] Batch 97, loss = 0.142447
I0614 16:42:54.270200 8992 caffe.cpp:309] Batch 98, accuracy = 1
I0614 16:42:54.270200 8992 caffe.cpp:309] Batch 98, loss = 0.00371724
I0614 16:42:54.301453 8992 caffe.cpp:309] Batch 99, accuracy = 0.99
I0614 16:42:54.301453 8992 caffe.cpp:309] Batch 99, loss = 0.0101131
I0614 16:42:54.301453 8992 caffe.cpp:314] Loss: 0.0295659
I0614 16:42:54.301453 8992 caffe.cpp:326] accuracy = 0.99
I0614 16:42:54.301453 8992 caffe.cpp:326] loss = 0.0295659 (* 1 = 0.0295659 loss)
C:\DeepLearning\caffe-master>
```

获得99%精度的预测结果

- Caffe简介与安装
- 手写字识别深度模型训练与测试
- 结合OpenCV的手写字识别系统

- 结合OpenCV的手写字识别系统

OpenCV

+

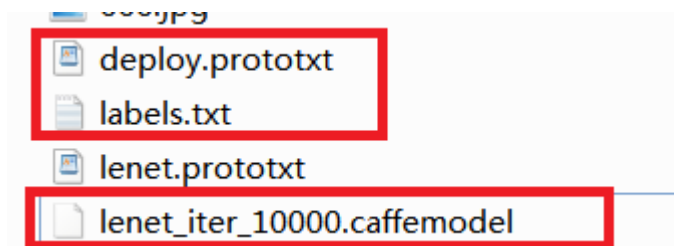
Caffe

图像处理+接口+界面+系统实现

分类

- 结合OpenCV的手写字识别系统

准备三个文件

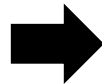


●结合OpenCV的手写字识别系统

```

name: "LeNet"
layer {
  name: "mnist"
  type: "Data"
  top: "data"
  top: "label"
  include {
    phase: TRAIN
  }
  transform_param {
    scale: 0.00390625
  }
  data_param {
    source: "examples/mnist/mnist_train_lmdb"
    batch_size: 64
    backend: LMDB
  }
}
layer {
  name: "mnist"
  type: "Data"
  top: "data"
  top: "label"
  include {
    phase: TEST
  }
  transform_param {
    scale: 0.00390625
  }
  data_param {
    source: "examples/mnist/mnist_test_lmdb"
    batch_size: 100
    backend: LMDB
  }
}
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  param {
    lr_mult: 1
  }
  param {
    lr_mult: 2
  }
}

```



```

name: "LeNet"
layer {
  name: "data"
  type: "Input"
  top: "data"
  input_param { shape: { dim: 64 dim: 1 dim: 28 dim: 28 } }
}
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  param {
    lr_mult: 1
  }
  param {
    lr_mult: 2
  }
}

```

deploy.prototxt

lenet_train_test.prototxt

●结合OpenCV的手写字识别系统

```
inner_product_param {
  num_output: 10
  weight_filler {
    type: "xavier"
  }
  bias_filler {
    type: "constant"
  }
}
}
layer {
  name: "accuracy"
  type: "Accuracy"
  bottom: "ip2"
  bottom: "label"
  top: "accuracy"
  include {
    phase: TEST
  }
}
}
layer {
  name: "loss"
  type: "SoftmaxWithLoss"
  bottom: "ip2"
  bottom: "label"
  top: "loss"
}
}
```

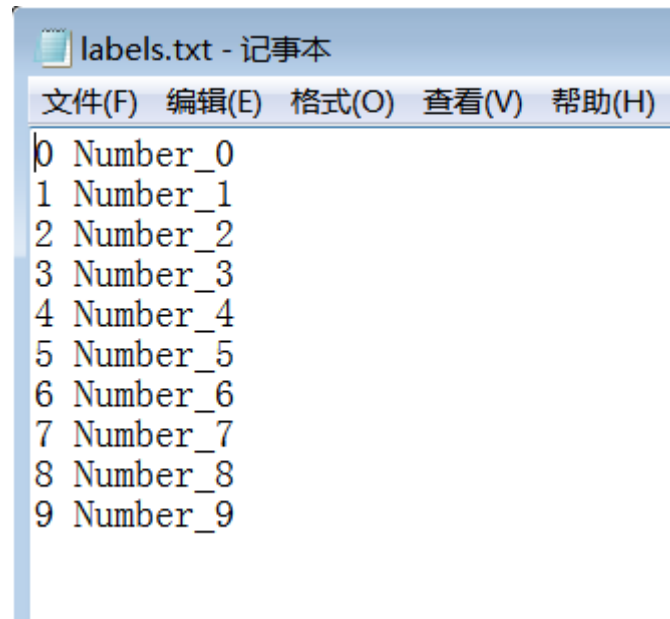


```
inner_product_param {
  num_output: 10
  weight_filler {
    type: "xavier"
  }
  bias_filler {
    type: "constant"
  }
}
}
layer {
  name: "prob"
  type: "Softmax"
  bottom: "ip2"
  top: "prob"
}
}
```

deploy.prototxt

lenet_train_test.prototxt

- 结合OpenCV的手写字识别系统



```
labels.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
0 Number_0
1 Number_1
2 Number_2
3 Number_3
4 Number_4
5 Number_5
6 Number_6
7 Number_7
8 Number_8
9 Number_9
```

●结合OpenCV的手写字识别系统

```
int main()
{
    CV_TRACE_FUNCTION();
    //读取模型参数和模型结构文件
    //String modelTxt = "lenet.prototxt";
    String modelTxt = "deploy.prototxt";
    String modelBin = "lenet_iter_10000.caffemodel";
    //读取图片
    String imageFile = "66.jpg";
    //合成网络
    Net net = dnn::readNetFromCaffe(modelTxt, modelBin);
    //判断网络是否生成成功
    if (net.empty())
    {
        std::cerr << "Can't load network by using the following files:
" << std::endl;
        exit(-1);
    }
    cerr << "net read successfully" << endl;

    //读取图片
    Mat img = imread(imageFile, 0); //这个很重要，必须要加0，否则变成3通道，会错误!!!
    //printf("nchannel=%d\n", img->nChannels);
    Mat img_new;
    img_new.create(img.rows, img.cols, img.type());
    //对图像进行二值化处理
    threshold(img, img_new, 100, 255, THRESH_BINARY);
    //img.copyTo(img_new);
```

```
cerr << "image read successfully" << endl;
/* Mat inputBlob = blobFromImage(img, 1, Size(224, 224),
Scalar(104, 117, 123)); */
//构造blob，为传入网络做准备，图片不能直接进入网络
//Mat inputBlob = blobFromImage(img_new, 1, Size(28, 28));
Mat inputBlob = blobFromImage(img_new, 1.0/255, Size(28, 28)); // 必须归一化

Mat prob;
cv::TickMeter t;
for (int i = 0; i < 1; i++) //for (int i = 0; i < 10; i++)
{
    //printf("here3\n");
    CV_TRACE_REGION("forward");
    //将构建的blob传入网络data层
    //printf("here4\n");
    net.setInput(inputBlob, "data");
    //printf("here5\n");
    //计时
    t.start();
    //前向预测
    //printf("here6\n");
    prob = net.forward("prob");
    //printf("here7\n");
    //停止计时
    t.stop();
}

int classId;
double classProb;
//找出最高的概率ID存储在classId，对应的标签在classProb中
getMaxClass(prob, &classId, &classProb);
//打印出结果
std::vector<String> classNames = readClassNames();
std::cout << "Best class: #" << classId << " " << classNames.at(classId) << " " << std::endl;
std::cout << "Probability: " << classProb * 100 << "%" << std::endl;
```

●结合OpenCV的手写字识别系统

```
static void mouse_call_fun( int event, int x, int y, int flags ,void* param)
{
    if( Displmg.empty() )
        return;
    //
    switch( event )
    {
        case CV_EVENT_LBUTTONDOWN://输入一个点
            CurrentPosition = Point(x,y);

            printf("当前点坐标: (%d,%d)\n",CurrentPosition.x,CurrentPosition.y);

            if(Control==0)//若原来处于空状态, 就变为写状态
            {
                Control=1;
                printf("鼠标左键开启写字状态\n");
            }
            break;
        case CV_EVENT_LBUTTONUP:
            printf("释放左键\n");
            if(Control==1)//若原来处于写字状态, 就变停止写字
            {
```

```
do
{
    //这里做个标记, 什么状态, 就向什么窗口进行输入
    imshow("Board",Displmg);

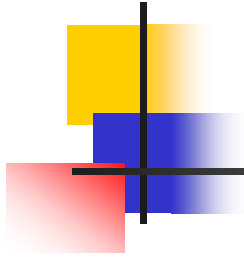
    key=waitKey(0);

    if(key=='1')
    {
        //分割操作

        //进行识别操作

    }
    else if(key=='w'||key=='W')//键盘开启写和关闭写的状态
    {
        printf("\n");
    }
    else if(key=='2')//等于2则离开
    {
        printf("leave\n");
        break;
    }
} while (1);
```

界面控制部分程序（参考HandWritingBoard程序）



请同学们上机实习